

LEAFGUARD - PLANT DISEASE DETECTION SYSTEM

Project Report Submitted in the partial fulfillment of the capstone project Of

Bachelor of Computer Application

By:

Abhay Chopra

Roll no: 2303004001

Academic Session (2023-2026)

Under the Guidance of

Adil Raja

Archana Salaria

Assistant Professor

Associate Professor

Department Of Computer Science And Engineering

Rayat Bahra University , Kharar

Punjab, India

CERTIFICATE

This is to certify that the project titled **“LeafGuard: Plant Disease Detection System”** has been completed and submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application. The project has been carried out with sincere effort and demonstrates practical application of computer vision, machine learning, database logging, and web application development.

The work presented in this report is based on the implementation of a Flask-based application that accepts plant leaf images, preprocesses them, classifies plant diseases using an EfficientNetB0 deep learning model, and presents actionable treatment guidance to the user. The project is suitable for academic evaluation because it combines theoretical machine learning concepts with a working user-facing software system.

Project Guide Signature: _____

Department Seal: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guides, mentors, classmates, and all those who supported the completion of this project. Their guidance helped me understand the practical connection between programming, machine learning, image processing, and real-world agricultural problems.

I am especially thankful for the academic environment that encouraged experimentation with image preprocessing, EfficientNetB0-based classification, Flask routing, SQLite storage, and responsive interface design. The project improved my understanding of how a machine learning model becomes useful only when it is connected to a complete workflow that ordinary users can access.

I also acknowledge the importance of open learning resources and publicly available plant disease datasets that made it possible to study disease classes, symptoms, treatment recommendations, and model behavior. This report is the result of continuous learning, testing, documentation, and refinement.

DECLARATION

I hereby declare that the project titled **“LeafGuard: Plant Disease Detection System”** is an original academic project prepared as part of my capstone work. The design, implementation, analysis, and documentation have been completed with sincere effort and academic integrity.

The system described in this report has been developed using Python, Flask, OpenCV, TensorFlow/Keras, EfficientNetB0, SQLite, HTML, CSS, and JavaScript. Wherever external libraries, datasets, or conceptual references have been used, they have been acknowledged appropriately in the report.

I take responsibility for the correctness of the work described here and confirm that the project has not been submitted elsewhere for the award of any other degree, diploma, or certificate.

Student Signature: _____

Date: _____

Index:

Chapter 1: Introduction

- Overview of LeafGuard
- Background and Motivation
- Importance of Plant Disease Detection
- Problem Statement and Objectives

Chapter 2: Project Plan and Requirements Analysis

- Agile Planning and Development Phases
- Functional and Non-Functional Requirements
- Hardware, Software, and Dataset Requirements

Chapter 3: System Design and Architecture

- Overall Architecture
- Data Flow and Module Design
- Database Design
- User Interface Design

Chapter 4: Implementation and Results

- OpenCV Preprocessing Pipeline
- EfficientNetB0 Prediction Service
- Flask API and Dashboard
- Result Interpretation and Treatment Advice

Chapter 5: Testing and Evaluation

- Unit Testing, Integration Testing, and UI Testing
- Model Testing and Error Handling
- Evaluation Observations

Chapter 6: Conclusion

- Summary of Achievements
- Skills Learned and Academic Impact

Chapter 7: Future Scope

- Deployment, Model Improvement, and Farmer-Focused Features

Appendix A: Source Code

References

Chapter 1: INTRODUCTION

1.1 Overview of the Project

LeafGuard is a plant disease detection system designed to identify common crop diseases from leaf images. The application combines image preprocessing, deep learning classification, and a web-based interface so that users can upload a leaf image and receive disease information, confidence score, likely symptoms, severity status, and treatment guidance.

The system uses a Flask backend to manage image upload, validation, preprocessing, prediction, and response generation. Image preprocessing is used to resize and prepare the leaf image before it is passed into an EfficientNetB0-based model. The prediction is then mapped to disease labels, symptom lists, and treatment advice stored in the application configuration.

The project demonstrates how artificial intelligence can support agriculture by making early disease recognition more accessible. Although the system is academic in scale, its design follows a realistic workflow: input acquisition, image preparation, model inference, result explanation, and record keeping through a database-backed dashboard.

1.2 Background and Motivation

Agriculture is one of the most important sectors for food security, rural employment, and economic stability. Plant diseases reduce crop yield, lower market quality, and increase the cost of cultivation. Many farmers identify diseases only after visible damage has spread, which makes treatment more expensive and less effective.

Advances in computer vision and deep learning have made it possible to classify plant diseases from images with increasing accuracy. A smartphone or web-based upload interface can become a practical bridge between farmers and machine learning models. LeafGuard is motivated by this possibility: a simple tool that converts a leaf image into useful decision support.

The project is also motivated by the need for students to understand complete software systems rather than isolated algorithms. A trained model alone is not enough. It must be connected to preprocessing, routing, validation, storage, interface design, and error handling before it can solve a real problem.

1.3 Importance of Plant Disease Detection

Early disease detection helps reduce the spread of infection, protects crop productivity, and supports responsible use of pesticides and fungicides.

Automated analysis can assist users in areas where expert plant pathologists or agricultural advisors are not immediately available.

A digital record of predictions can help track disease patterns over time and support better field-level decision making.

Computer vision systems create opportunities for scalable, low-cost agricultural advisory services.

1.4 Problem Statement

The main problem addressed by LeafGuard is the difficulty of quickly identifying plant diseases from visible leaf symptoms. Manual identification requires experience, and incorrect diagnosis may lead to wrong

treatment, crop loss, or unnecessary chemical use. The project therefore aims to provide an accessible image-based system for disease prediction and treatment guidance.

The system accepts common image formats, applies preprocessing to make the input more suitable for classification, predicts the most likely class, and presents the output in a user-friendly format. The output is not intended to replace professional agricultural advice, but it provides a helpful first-level screening mechanism.

1.5 Objectives

- To design and develop a web application for plant leaf image upload and disease prediction.
- To preprocess images using OpenCV operations such as resizing, denoising, color conversion, masking, and morphological filtering.
- To integrate an EfficientNetB0 deep learning model for multiclass plant disease classification.
- To provide confidence score, symptoms, severity label, and treatment advice with every prediction.
- To store prediction history in SQLite and display summary statistics through a dashboard.
- To create a responsive and visually clear interface for users.

1.6 Scope of the Project

The project covers 38 PlantVillage-style classes across crops such as apple, corn, grape, potato, tomato, strawberry, pepper, peach, cherry, squash, orange, soybean, raspberry, and blueberry. It supports image upload through a browser interface and produces structured JSON responses for prediction results.

The scope includes image preprocessing, backend inference, treatment mapping, symptom display, database logging, and dashboard reporting. It does not include real-time field camera integration, mobile app packaging, cloud deployment, or expert-verified diagnosis, which are discussed later under future scope.

Chapter 2: PROJECT PLAN & REQUIREMENTS ANALYSIS

2.1 Development Methodology

The project follows an incremental development approach. Each module was planned, implemented, tested, and connected to the next module. This approach made it easier to isolate errors and improve the system gradually.

The first phase focused on understanding the problem domain and selecting the dataset and model architecture. The second phase implemented preprocessing and model service logic. The third phase connected the model to Flask routes. The fourth phase developed the user interface and dashboard. The final phase involved testing, documentation, and report preparation.

2.2 Development Phases

- Phase 1: Dataset study and disease class identification.
- Phase 2: OpenCV preprocessing pipeline design.
- Phase 3: EfficientNetB0 training and model export.
- Phase 4: Flask application integration.
- Phase 5: SQLite prediction logging and dashboard creation.
- Phase 6: Interface polishing, testing, and documentation.

2.3 Functional Requirements

- The user shall be able to upload a JPG, JPEG, or PNG leaf image.
- The system shall reject unsupported files with a clear error message.
- The system shall preprocess the uploaded image before model prediction.
- The system shall predict the disease class and confidence score.
- The system shall show symptoms, severity, model name, processing method, and dataset name.
- The system shall store each prediction in a database.
- The dashboard shall display prediction history and basic statistics.

2.4 Non-Functional Requirements

- Usability: The interface must be simple enough for non-technical users.
- Reliability: The system must handle missing files, unsupported formats, and missing model files gracefully.
- Performance: Preprocessing and prediction should complete quickly for standard leaf images.
- Maintainability: Disease classes, symptoms, and treatments should be stored in configuration mappings.
- Scalability: The design should allow future deployment, model replacement, and additional disease classes.
- Security: Uploaded filenames should be sanitized before saving to the server.

Requirement Area	Description
Frontend	HTML templates, CSS styling, upload form, prediction result panel, and dashboard screens.
Backend	Flask routes for index, dashboard, uploads, and prediction API.
Image Processing	OpenCV resizing, denoising, HSV masking, morphology, and RGB conversion.
Machine Learning	TensorFlow/Keras EfficientNetB0 model for multiclass disease classification.
Database	SQLite table for prediction logs containing image, disease, confidence, advice, and timestamp.
Dataset	PlantVillage-style disease classes represented through the application class list.

2.5 Software and Hardware Requirements

The software stack includes Python, Flask, OpenCV, NumPy, TensorFlow/Keras, SQLite, HTML, CSS, and JavaScript. The application can run on a standard development machine with sufficient memory to load the model and process images.

Recommended hardware includes a modern multicore processor, 8 GB RAM, and optional GPU support for model training. For inference, CPU execution is acceptable for academic demonstration because EfficientNetB0 offers a practical balance between accuracy and model size.

Chapter 3: SYSTEM DESIGN & ARCHITECTURE

3.1 System Architecture

LeafGuard is organized into clear modules. The browser interface sends the uploaded image to the Flask backend. The backend validates and saves the file, passes it to the preprocessing pipeline, sends the processed image to the model service, maps the predicted class to symptoms and treatments, stores the prediction record, and returns a structured result to the frontend.

This separation of concerns keeps the project maintainable. The preprocessing module does not know about Flask routes, the model service does not know about the database, and the database logger does not know about HTML templates. Each part performs a focused responsibility.

3.2 Data Flow

- Step 1: User opens the web application and selects a leaf image.
- Step 2: The browser submits the image to the `/predict` route.
- Step 3: Flask validates the file extension and saves the upload.
- Step 4: OpenCV preprocessing resizes, denoises, masks, and converts the image.
- Step 5: EfficientNetB0 predicts the most probable disease class.
- Step 6: The application maps the class to symptoms, severity, and treatment advice.
- Step 7: SQLite stores the prediction record.
- Step 8: The frontend displays a user-friendly result.

Module	Responsibility	Main File
Flask App	Routes, upload validation, prediction endpoint, response formatting.	src/app.py
Preprocessing	Image resizing, denoising, HSV mask generation, morphology, RGB conversion.	src/preprocessing/pipeline.py
Inference	Model loading, normalization, batch creation, probability interpretation.	src/inference/model_service.py
Database	Prediction storage and dashboard statistics.	src/db/logger.py
Configuration	Class names, symptom mapping, and treatment mapping.	src/config.py
Interface	Upload page, dashboard page, and styling.	templates/, static/

3.3 Image Preprocessing Design

The preprocessing pipeline prepares uploaded leaf images for model inference. Every image is resized to 224 x 224 pixels because EfficientNetB0 commonly expects fixed-size input. The current training setup uses normalized color images so that training and prediction remain consistent.

The pipeline converts the image into HSV color space and applies a green-range mask to isolate the leaf region. Morphological opening and closing help remove small noise and fill gaps in the mask. A Gaussian blur smooths the mask before bitwise masking is applied. Finally, the image is converted from BGR to RGB for compatibility with common deep learning workflows.

3.4 Database Design

The SQLite database stores prediction logs in a single table named `prediction\_logs`. Each row contains the uploaded image name, predicted disease name, confidence score, treatment advice, and timestamp. This structure is sufficient for the current academic version and can be extended later with user accounts, location, crop season, or field identifiers.

Field	Type	Purpose
id	INTEGER PRIMARY KEY	Unique identifier for every prediction.
image_name	TEXT	Stores uploaded image filename.
disease_name	TEXT	Stores predicted class label.
confidence	REAL	Stores confidence percentage.
treatment_advice	TEXT	Stores recommended treatment message.
created_at	TIMESTAMP	Records prediction time automatically.

3.5 User Interface Design

The interface is designed around a direct workflow: upload, analyze, view result, and inspect history. The upload page presents the core prediction experience, while the dashboard supports review of past predictions. CSS styling gives the project a coherent LeafGuard identity with plant-focused colors and clear result cards.

The user interface avoids unnecessary technical complexity. Instead of exposing raw model probabilities for every class, the application shows the most important information: plant name, disease name, confidence, severity, symptoms, treatment advice, model, processing method, and dataset.

Chapter 4: IMPLEMENTATION & RESULTS

4.1 Backend Implementation

The backend is implemented in Flask. The application initializes upload paths, model paths, preprocessing service, prediction logger, and model service. If the trained model file exists, it is loaded through the `PlantDiseaseModelService`. If the model file is missing, the prediction endpoint returns a clear server-side error explaining the expected model path.

The `/predict` route is the central backend route. It obtains the uploaded file, validates it, saves it securely, preprocesses the image, runs inference, logs the result, and returns a JSON object. This route demonstrates a complete request-response cycle for machine-learning-enabled web software.

4.2 Preprocessing Implementation

The `LeafPreprocessor` class contains a configurable preprocessing pipeline. The use of a dataclass for preprocessing parameters makes the code readable and adjustable. Target image size, denoising strength, template window size, search window size, blur kernel, and morphology kernel can be modified without rewriting the processing logic.

The segmentation logic uses HSV thresholds because color-based masking is often easier in HSV than in raw BGR. The selected green range captures leaf regions while excluding part of the background. This step helps the classifier focus more on leaf tissue and visible disease patterns.

4.3 Model Service Implementation

The model service loads the trained Keras model and stores the class names and treatment map. During prediction, the preprocessed RGB image is converted to float values in the range 0 to 1 and expanded into a batch dimension. The model output is interpreted using `argmax`, and the highest-probability class becomes the prediction.

The service returns a structured `PredictionResult` containing the disease name, confidence percentage, and treatment advice. This makes the calling Flask route cleaner because prediction details are encapsulated in one object.

4.4 Frontend Implementation

The frontend includes an upload-focused index page and a dashboard page. The index page allows users to select a leaf image and view the prediction result without needing to understand the backend. The dashboard page presents stored prediction logs and basic statistics from the SQLite database.

The CSS file provides visual polish through structured layout, result panels, buttons, and plant-themed styling. The interface is important because a disease detection system must be understandable to users who may not have a technical background.



Figure 4.1: Sample leaf images used for interface demonstration.

4.5 Disease Class Coverage

The configuration contains 38 disease and healthy classes. A wide class list makes the system more useful because it is not limited to a single crop. The project includes healthy categories as well as disease categories, allowing the interface to return positive care messages when no disease is detected.

S. No.	Disease / Health Class
1	Apple scab
2	Apple black rot
3	Cedar apple rust
4	Apple healthy
5	Blueberry healthy
6	Cherry healthy
7	Cherry powdery mildew
8	Corn gray leaf spot
9	Corn common rust
10	Corn healthy
11	Corn northern leaf blight
12	Grape black rot
13	Grape esca
14	Grape healthy
15	Grape leaf blight
16	Orange citrus greening
17	Peach bacterial spot
18	Peach healthy
19	Bell pepper bacterial spot
20	Bell pepper healthy
21	Potato early blight
22	Potato healthy
23	Potato late blight
24	Raspberry healthy
25	Soybean healthy
26	Squash powdery mildew
27	Strawberry healthy
28	Strawberry leaf scorch
29	Tomato bacterial spot
30	Tomato early blight

S. No.	Disease / Health Class
31	Tomato healthy
32	Tomato late blight
33	Tomato leaf mold
34	Tomato septoria leaf spot
35	Tomato spider mites
36	Tomato target spot
37	Tomato mosaic virus
38	Tomato yellow leaf curl virus

4.6 Result Format

A successful prediction returns a result object containing disease name, raw class name, plant name, confidence, severity, symptoms, treatment, model name, processing method, dataset name, and uploaded image name. This result format is useful because it is both human-readable and machine-readable.

The system derives a plant name and a simplified disease name by splitting PlantVillage-style class labels around the triple underscore marker. This makes labels more readable on the frontend while still preserving the raw class name for debugging or record keeping.

Chapter 5: TESTING & EVALUATION

5.1 Testing Strategy

Testing was planned around the main risk points of the system: file upload validation, preprocessing correctness, model availability, prediction response structure, database logging, and dashboard display. Each risk point was evaluated from the perspective of both developer correctness and user experience.

Because the system depends on a trained model file, backend error handling is important. The application explicitly checks for the model path and returns a meaningful message if the model is not present. This prevents silent failure and helps during setup.

Test Area	Test Case	Expected Result
Upload	Submit without selecting a file.	Application returns an error asking for a leaf image.
Upload	Submit unsupported file extension.	Application rejects the file.
Preprocessing	Process valid JPG/PNG image.	Image is resized, denoised, segmented, and converted to RGB.
Model	Run prediction with valid model.	System returns class and confidence.
Database	Complete a prediction.	Prediction row is inserted into SQLite table.
Dashboard	Open dashboard after predictions.	History and statistics are displayed.

5.2 Unit Testing Considerations

The preprocessing module can be tested by passing a valid NumPy image array and confirming that the output shape is 224 x 224 x 3. It can also be tested with invalid file paths to confirm that the correct exception is raised.

The database logger can be tested by creating a temporary SQLite database, inserting sample predictions, and checking whether ``get_all_predictions`` and ``get_stats`` return the expected values. These tests are useful because they do not require a trained model.

5.3 Integration Testing

Integration testing checks whether the upload route, preprocessing pipeline, model service, and database logger work together. The most important integration test is a complete prediction flow using a valid leaf image. The expected result is a JSON response with disease details and a corresponding record in the database.

Another integration test is the dashboard route. After predictions are made, the dashboard should fetch all stored rows and show summary statistics. This confirms that backend data is visible through the frontend layer.

5.4 Evaluation Observations

The system architecture is suitable for academic demonstration because the modules are readable and the workflow is complete. The preprocessing approach is lightweight and deterministic. The prediction service is simple, which makes it easy to replace the model file in future iterations.

Accuracy depends on the trained EfficientNetB0 model, the quality of training data, and how closely uploaded real-world images match the dataset distribution. Field images may include complex backgrounds, lighting variation, occlusion, multiple leaves, soil, or shadows. These factors can reduce prediction reliability,

so future versions should include stronger augmentation and real-field validation.

5.5 Limitations

- The system requires a trained `.keras` model file before prediction can run.
- The HSV green mask may not perfectly segment leaves under all lighting conditions.
- The application does not yet support user authentication or farm-level record grouping.
- Treatment advice is rule-based and should be reviewed by agricultural experts before production use.
- The system is web-based and is not yet packaged as an offline mobile app.

Chapter 6: CONCLUSION

6.1 Summary of Achievements

LeafGuard successfully demonstrates an end-to-end plant disease detection workflow. The project accepts uploaded leaf images, preprocesses them, predicts disease classes using an EfficientNetB0 model, maps predictions to symptoms and treatment advice, stores prediction history in SQLite, and displays results through a Flask web interface.

The project is meaningful because it connects artificial intelligence with agriculture, a domain where early detection and timely action can protect yield and reduce losses. It also shows how academic software projects can become practical when they are designed as complete systems rather than isolated code fragments.

6.2 Academic Learning

The development of LeafGuard provided practical experience in Python programming, computer vision, machine learning integration, backend routing, database design, template rendering, and frontend styling. It also improved understanding of software modularity, configuration-driven design, and user-centered result presentation.

The project bridges classroom concepts with real implementation. Concepts such as convolutional neural networks, image normalization, HTTP requests, database persistence, and error handling become clearer when they are implemented together in one working application.

6.3 Final Conclusion

Overall, LeafGuard is a strong academic demonstration of machine-learning-assisted agricultural diagnosis. It is not a replacement for expert inspection, but it provides a useful first-level screening tool that can be expanded into a more advanced advisory platform. With further model improvement, field testing, mobile deployment, and expert-verified treatment recommendations, the system can become a practical tool for farmers, students, and agricultural support teams.

Chapter 7: FUTURE SCOPE

7.1 Improved Model Accuracy

Future versions can train the model with larger and more diverse datasets that include real field images, different lighting conditions, various camera qualities, and multiple disease stages. Stronger data augmentation can improve robustness.

The system can also compare EfficientNetB0 with ResNet, DenseNet, Vision Transformers, or ensemble models. Evaluation metrics such as precision, recall, F1-score, confusion matrix, and class-wise accuracy should be included in future experiments.

7.2 Mobile Application

A mobile app would make LeafGuard more accessible to farmers and field workers. The mobile version could support camera capture, offline image storage, GPS-based tagging, local language instructions, and simplified treatment workflows.

7.3 Cloud Deployment

Deploying the application on cloud infrastructure would allow remote access, centralized model updates, and scalable usage. Cloud deployment can include Docker containers, API gateways, managed databases, monitoring, and automated backups.

7.4 Expert Recommendation System

The current treatment map provides simple guidance. Future versions can include an expert-verified recommendation engine that considers crop type, disease stage, season, local climate, organic or chemical treatment preference, and severity level.

7.5 Multilingual Support

Since the target users may include farmers from different regions, multilingual support would make the application more inclusive. Disease names, symptoms, and treatment advice can be translated into Hindi, Punjabi, and other regional languages.

7.6 Analytics Dashboard

A more advanced dashboard can show disease trends, prediction counts by crop, high-risk disease categories, confidence distributions, and seasonal changes. If location data is added, the system could display heatmaps of common plant diseases.

7.7 IoT and Field Monitoring

LeafGuard can be integrated with IoT sensors that monitor humidity, temperature, rainfall, and soil moisture. Combining image-based disease detection with environmental data can improve disease risk prediction and preventive recommendations.

7.8 Production Security

Future production versions should include user authentication, file size limits, virus scanning, secure upload storage, role-based dashboards, HTTPS deployment, and database backup policies. These improvements are important if the system is made public.

REFERENCES

- TensorFlow and Keras documentation for model loading and inference workflows.
- OpenCV documentation for resizing, denoising, HSV conversion, masking, morphology, and image processing.
- Flask documentation for routing, request handling, templates, and static file serving.
- SQLite documentation for lightweight relational database storage.
- PlantVillage dataset references for plant disease image classification research.
- EfficientNetB0 research and transfer learning references for efficient convolutional neural networks.
- Python documentation for pathlib, dataclasses, JSON handling, and application structure.

APPENDIX A: SOURCE CODE

A.1 Appendix Overview

This appendix includes the major source files used in the LeafGuard project. The code is included to document the implementation details behind preprocessing, model inference, Flask routing, configuration mapping, database logging, and interface design.

A.2 Flask Application - src/app.py

```
001 from __future__ import annotations
002
003 import json
004 import os
005 from functools import wraps
006 from pathlib import Path
007
008 from flask import Flask, jsonify, render_template, request, send_from_directory, session,
    redirect, url_for
009 from werkzeug.utils import secure_filename
010 from werkzeug.security import generate_password_hash, check_password_hash
011
012 from config import CLASS_NAMES, SYMPTOM_MAP, TREATMENT_MAP
013 from db.logger import PredictionLogger
014 from inference.model_service import PlantDiseaseModelService
015 from preprocessing.pipeline import LeafPreprocessor
016
017 BASE_DIR = Path(__file__).resolve().parent.parent
018 UPLOAD_DIR = BASE_DIR / "uploads"
019 MODEL_PATH = BASE_DIR / "models" / "plant_disease_efficientnetb0.keras"
020
021 ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png"}
022
023 app = Flask(__name__, template_folder=str(BASE_DIR / "templates"), static_folder=str(
    BASE_DIR / "static"))
024 app.secret_key = os.environ.get("FLASK_SECRET_KEY", os.urandom(24))
025 app.config["UPLOAD_FOLDER"] = str(UPLOAD_DIR)
026 UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
027
028 def login_required(f):
029     @wraps(f)
030     def decorated_function(*args, **kwargs):
031         if "user_id" not in session:
032             return redirect(url_for("login"))
033         return f(*args, **kwargs)
034     return decorated_function
035
036 preprocessor = LeafPreprocessor()
037 prediction_logger = PredictionLogger(BASE_DIR / "predictions.db")
038
039 model_service = None
040 if MODEL_PATH.exists():
041     class_names_path = BASE_DIR / "models" / "class_names.json"
042     class_names = CLASS_NAMES
043     if class_names_path.exists():
044         class_names = json.loads(class_names_path.read_text(encoding="utf-8"))
045     model_service = PlantDiseaseModelService(MODEL_PATH, class_names, TREATMENT_MAP)
046
047
048 def _is_allowed(filename: str) -> bool:
```

A.2 Flask Application - src/app.py (continued)

```
049     return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS
050
051
052 @app.route("/login", methods=["GET", "POST"])
053 def login():
054     if "user_id" in session:
055         return redirect(url_for("index"))
056     error = None
057     if request.method == "POST":
058         username = request.form.get("username")
059         password = request.form.get("password")
060         user = prediction_logger.get_user_by_username(username)
061         if user and check_password_hash(user["password_hash"], password):
062             session["user_id"] = user["id"]
063             session["username"] = user["username"]
064             return redirect(url_for("index"))
065         else:
066             error = "Invalid username or password"
067     return render_template("login.html", error=error)
068
069 @app.route("/signup", methods=["GET", "POST"])
070 def signup():
071     if "user_id" in session:
072         return redirect(url_for("index"))
073     error = None
074     if request.method == "POST":
075         username = request.form.get("username")
076         password = request.form.get("password")
077         if not username or not password:
078             error = "Username and password are required"
079         else:
080             password_hash = generate_password_hash(password)
081             if prediction_logger.create_user(username, password_hash):
082                 user = prediction_logger.get_user_by_username(username)
083                 session["user_id"] = user["id"]
084                 session["username"] = user["username"]
085                 return redirect(url_for("index"))
086             else:
087                 error = "Username already exists"
088     return render_template("signup.html", error=error)
089
090 @app.route("/logout")
091 def logout():
092     session.clear()
093     return redirect(url_for("login"))
094
095 @app.route("/", methods=["GET", "POST"])
096 @login_required
```

A.2 Flask Application - src/app.py (continued)

```
097 def index():
098     return render_template("index.html")
099
100 @app.route("/dashboard")
101 @login_required
102 def dashboard():
103     predictions = prediction_logger.get_all_predictions(session["user_id"])
104     stats = prediction_logger.get_stats(session["user_id"])
105     return render_template("dashboard.html", predictions=predictions, stats=stats)
106
107
108 @app.route("/uploads/<filename>")
109 def uploaded_file(filename):
110     return send_from_directory(app.config["UPLOAD_FOLDER"], filename)
111
112
113 @app.route("/predict", methods=["POST"])
114 @login_required
115 def predict():
116     file = request.files.get("leaf_image")
117     if not file or file.filename == "":
118         return jsonify({"error": "Please upload a leaf image file."}), 400
119
120     if not _is_allowed(file.filename):
121         return jsonify({"error": "Only JPG, JPEG, and PNG files are supported."}), 400
122
123     if model_service is None:
124         return jsonify({"error": "Model file not found at models/
plant_disease_efficientnetb0.keras."}), 500
125
126     filename = secure_filename(file.filename)
127     image_path = UPLOAD_DIR / filename
128     file.save(image_path)
129
130     preprocessed = preprocessor.preprocess_from_path(image_path)
131     result = model_service.predict(preprocessed)
132
133     prediction_logger.log_prediction(
134         user_id=session["user_id"],
135         image_name=filename,
136         disease_name=result.disease_name,
137         confidence=result.confidence,
138         treatment_advice=result.treatment_advice,
139     )
140
141     disease_name = result.disease_name
142     if "___" in disease_name:
143         plant_name, short_disease = disease_name.split("___", 1)
144     else:
```

A.2 Flask Application - src/app.py (continued)

```
145         plant_name, short_disease = "Plant", disease_name
146
147         symptoms = SYMPTOM_MAP.get(
148             disease_name,
149             ["Visible leaf discoloration", "Localized spots", "Potential disease
150             progression"],
151             )
152         sev_class = "low" if "healthy" in disease_name.lower() else ("high" if result.
153             confidence >= 85 else "med")
154         sev_label = "Healthy" if sev_class == "low" and "healthy" in disease_name.lower()
155         else ("High" if sev_class == "high" else "Medium")
156
157         return jsonify(
158             {
159                 "disease_name": short_disease.replace("_", " "),
160                 "raw_class_name": disease_name,
161                 "plant_name": plant_name.replace("_", " "),
162                 "confidence": round(result.confidence, 2),
163                 "severity": sev_label,
164                 "severity_class": sev_class,
165                 "symptoms": symptoms,
166                 "treatment": [result.treatment_advice],
167                 "model": "EfficientNetB0",
168                 "processing": "OpenCV + CNN",
169                 "dataset": "PlantVillage",
170                 "image_name": filename,
171             }
172         )
173
174     if __name__ == "__main__":
175         app.run(debug=True)
```

A.3 Preprocessing Pipeline - src/preprocessing/pipeline.py

```
001 from __future__ import annotations
002
003 from dataclasses import dataclass
004 from pathlib import Path
005
006 import cv2
007 import numpy as np
008
009
010 @dataclass
011 class PreprocessConfig:
012     target_size: tuple[int, int] = (224, 224)
013     use_leaf_segmentation: bool = False
014     denoise_h: int = 10
015     denoise_template_window_size: int = 7
016     denoise_search_window_size: int = 21
017     blur_kernel_size: tuple[int, int] = (5, 5)
018     morph_kernel_size: tuple[int, int] = (5, 5)
019
020
021 class LeafPreprocessor:
022     def __init__(self, config: PreprocessConfig | None = None) -> None:
023         self.config = config or PreprocessConfig()
024
025     def preprocess(self, image_bgr: np.ndarray) -> np.ndarray:
026         resized = cv2.resize(image_bgr, self.config.target_size, interpolation=cv2.
INTER_AREA)
027
028         if not self.config.use_leaf_segmentation:
029             return cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
030
031         denoised = cv2.fastNlMeansDenoisingColored(
032             resized,
033             None,
034             h=self.config.denoise_h,
035             hColor=self.config.denoise_h,
036             templateWindowSize=self.config.denoise_template_window_size,
037             searchWindowSize=self.config.denoise_search_window_size,
038         )
039
040         segmented = self._segment_leaf(denoised)
041         rgb = cv2.cvtColor(segmented, cv2.COLOR_BGR2RGB)
042         return rgb
043
044     def preprocess_from_path(self, image_path: str | Path) -> np.ndarray:
045         image = cv2.imread(str(image_path))
046         if image is None:
047             raise FileNotFoundError(f"Could not read image: {image_path}")
048         return self.preprocess(image)
```

A.3 Preprocessing Pipeline - src/preprocessing/pipeline.py (continued)

```
049
050     def _segment_leaf(self, image_bgr: np.ndarray) -> np.ndarray:
051         hsv = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2HSV)
052
053         lower_green = np.array([20, 20, 20], dtype=np.uint8)
054         upper_green = np.array([95, 255, 255], dtype=np.uint8)
055         mask = cv2.inRange(hsv, lower_green, upper_green)
056
057         kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, self.config.
morph_kernel_size)
058         mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
059         mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
060         mask = cv2.GaussianBlur(mask, self.config.blur_kernel_size, 0)
061
062         segmented = cv2.bitwise_and(image_bgr, image_bgr, mask=mask)
063         return segmented
```

A.4 Model Service - src/inference/model_service.py

```
001 from __future__ import annotations
002
003 from dataclasses import dataclass
004 from pathlib import Path
005
006 import numpy as np
007 from tensorflow.keras.models import load_model
008
009
010 @dataclass
011 class PredictionResult:
012     disease_name: str
013     confidence: float
014     treatment_advice: str
015
016
017 class PlantDiseaseModelService:
018     def __init__(self, model_path: str | Path, class_names: list[str], treatment_map:
019 dict[str, str]) -> None:
020         self.model = load_model(str(model_path))
021         self.class_names = class_names
022         self.treatment_map = treatment_map
023
024     def predict(self, preprocessed_rgb: np.ndarray) -> PredictionResult:
025         image = preprocessed_rgb.astype("float32") / 255.0
026         batch = np.expand_dims(image, axis=0)
027         probs = self.model.predict(batch, verbose=0)[0]
028
029         class_idx = int(np.argmax(probs))
030         confidence = float(probs[class_idx] * 100.0)
031         disease_name = self.class_names[class_idx]
032         treatment_advice = self.treatment_map.get(
033             disease_name,
034             "Isolate affected plant and consult agricultural expert for targeted
035 treatment.",
036         )
037
038     return PredictionResult(
039         disease_name=disease_name,
040         confidence=confidence,
041         treatment_advice=treatment_advice,
042     )
```

A.5 Database Logger - src/db/logger.py

```
001 from __future__ import annotations
002
003 import sqlite3
004 from pathlib import Path
005
006
007 class PredictionLogger:
008     def __init__(self, db_path: str | Path = "predictions.db") -> None:
009         self.db_path = str(db_path)
010         self._initialize_db()
011
012     def _initialize_db(self) -> None:
013         with sqlite3.connect(self.db_path) as conn:
014             conn.execute(
015                 """
016                 CREATE TABLE IF NOT EXISTS users (
017                     id INTEGER PRIMARY KEY AUTOINCREMENT,
018                     username TEXT UNIQUE NOT NULL,
019                     password_hash TEXT NOT NULL,
020                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
021                 )
022                 """
023             )
024             conn.execute(
025                 """
026                 CREATE TABLE IF NOT EXISTS prediction_logs (
027                     id INTEGER PRIMARY KEY AUTOINCREMENT,
028                     user_id INTEGER,
029                     image_name TEXT NOT NULL,
030                     disease_name TEXT NOT NULL,
031                     confidence REAL NOT NULL,
032                     treatment_advice TEXT NOT NULL,
033                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
034                     FOREIGN KEY(user_id) REFERENCES users(id)
035                 )
036                 """
037             )
038             try:
039                 conn.execute("ALTER TABLE prediction_logs ADD COLUMN user_id INTEGER")
040             except sqlite3.OperationalError:
041                 pass
042
043     def create_user(self, username: str, password_hash: str) -> bool:
044         try:
045             with sqlite3.connect(self.db_path) as conn:
046                 conn.execute("INSERT INTO users (username, password_hash) VALUES (?, ?)",
047                     (username, password_hash))
048                 return True
049         except sqlite3.IntegrityError:
```

A.5 Database Logger - src/db/logger.py (continued)

```
049         return False
050
051     def get_user_by_username(self, username: str) -> dict | None:
052         with sqlite3.connect(self.db_path) as conn:
053             conn.row_factory = sqlite3.Row
054             cursor = conn.cursor()
055             cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
056             row = cursor.fetchone()
057             return dict(row) if row else None
058
059     def log_prediction(self, user_id: int, image_name: str, disease_name: str,
060                       confidence: float, treatment_advice: str) -> None:
061         with sqlite3.connect(self.db_path) as conn:
062             conn.execute(
063                 """
064                 INSERT INTO prediction_logs (user_id, image_name, disease_name,
065                 confidence, treatment_advice)
066                 VALUES (?, ?, ?, ?, ?)
067                 """,
068                 (user_id, image_name, disease_name, confidence, treatment_advice),
069             )
070
071     def get_all_predictions(self, user_id: int) -> list[dict]:
072         with sqlite3.connect(self.db_path) as conn:
073             conn.row_factory = sqlite3.Row
074             cursor = conn.cursor()
075             cursor.execute("SELECT * FROM prediction_logs WHERE user_id = ? ORDER BY
076             created_at DESC", (user_id,))
077             rows = cursor.fetchall()
078             return [dict(row) for row in rows]
079
080     def get_stats(self, user_id: int) -> dict:
081         with sqlite3.connect(self.db_path) as conn:
082             cursor = conn.cursor()
083             cursor.execute("SELECT COUNT(*) FROM prediction_logs WHERE user_id = ?", (
084             user_id,))
085             total = cursor.fetchone()[0]
086
087             cursor.execute("SELECT disease_name, COUNT(*) as count FROM prediction_logs
088             WHERE user_id = ? GROUP BY disease_name ORDER BY count DESC LIMIT 5", (user_id,))
089             top_diseases = cursor.fetchall()
090
091             return {
092                 "total_predictions": total,
093                 "top_diseases": top_diseases
094             }
```

A.6 Configuration - src/config.py

```
001 CLASS_NAMES = [
002     "Apple__Apple_scab",
003     "Apple__Black_rot",
004     "Apple__Cedar_apple_rust",
005     "Apple__healthy",
006     "Blueberry__healthy",
007     "Cherry_(including_sour)__healthy",
008     "Cherry_(including_sour)__Powdery_mildew",
009     "Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot",
010     "Corn_(maize)__Common_rust_",
011     "Corn_(maize)__healthy",
012     "Corn_(maize)__Northern_Leaf_Blight",
013     "Grape__Black_rot",
014     "Grape__Esca_(Black_Measles)",
015     "Grape__healthy",
016     "Grape__Leaf_blight_(Isariopsis_Leaf_Spot)",
017     "Orange__Haunglongbing_(Citrus_greening)",
018     "Peach__Bacterial_spot",
019     "Peach__healthy",
020     "Pepper,_bell__Bacterial_spot",
021     "Pepper,_bell__healthy",
022     "Potato__Early_blight",
023     "Potato__healthy",
024     "Potato__Late_blight",
025     "Raspberry__healthy",
026     "Soybean__healthy",
027     "Squash__Powdery_mildew",
028     "Strawberry__healthy",
029     "Strawberry__Leaf_scorch",
030     "Tomato__Bacterial_spot",
031     "Tomato__Early_blight",
032     "Tomato__healthy",
033     "Tomato__Late_blight",
034     "Tomato__Leaf_Mold",
035     "Tomato__Septoria_leaf_spot",
036     "Tomato__Spider_mites Two-spotted_spider_mite",
037     "Tomato__Target_Spot",
038     "Tomato__Tomato_mosaic_virus",
039     "Tomato__Tomato_Yellow_Leaf_Curl_Virus",
040 ]
041
042 TREATMENT_MAP = {
043     "Apple__Apple_scab": "Apply fungicide and remove infected leaves to limit spread.",
044     "Apple__Black_rot": "Prune affected areas and improve air circulation around plants.",
045     "Apple__Cedar_apple_rust": "Use resistant varieties and fungicidal sprays during
early growth.",
046     "Apple__healthy": "No disease detected. Continue regular watering and nutrient
schedule.",
047     "Blueberry__healthy": "No disease detected. Maintain optimal soil pH and watering.",
048     "Cherry_(including_sour)__healthy": "No disease detected. Maintain regular care.",
```

A.6 Configuration - src/config.py (continued)

```
049     "Cherry_(including_sour)___Powdery_mildew": "Apply sulfur or neem oil. Ensure good
air circulation.",
050     "Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot": "Use crop rotation, clean
plowing, and apply appropriate fungicides.",
051     "Corn_(maize)___Common_rust_": "Apply fungicides early in the season and use rust-
resistant hybrids.",
052     "Corn_(maize)___healthy": "No disease detected. Maintain balanced fertilization.",
053     "Corn_(maize)___Northern_Leaf_Blight": "Plant resistant hybrids and apply fungicides
if lesions appear early.",
054     "Grape___Black_rot": "Apply fungicides at the right time and remove infected plant
debris.",
055     "Grape___Esca_(Black_Measles)": "Remove infected vines and apply protective pruning
wound treatments.",
056     "Grape___healthy": "No disease detected. Maintain proper viticulture practices.",
057     "Grape___Leaf_blight_(Isariopsis_Leaf_Spot)": "Use appropriate fungicides and
maintain good canopy management.",
058     "Orange___Haunglongbing_(Citrus_greening)": "Remove infected trees and control
psyllid vectors with insecticides.",
059     "Peach___Bacterial_spot": "Apply copper-based bactericides and avoid overhead
irrigation.",
060     "Peach___healthy": "No disease detected. Maintain regular orchard care.",
061     "Pepper,_bell___Bacterial_spot": "Use disease-free seeds, rotate crops, and apply
copper sprays.",
062     "Pepper,_bell___healthy": "No disease detected. Continue standard care.",
063     "Potato___Early_blight": "Apply fungicides, practice crop rotation, and maintain
plant vigor.",
064     "Potato___healthy": "No disease detected. Continue standard care.",
065     "Potato___Late_blight": "Use resistant varieties and apply appropriate fungicides
before infection spreads.",
066     "Raspberry___healthy": "No disease detected. Maintain regular care.",
067     "Soybean___healthy": "No disease detected. Maintain regular care.",
068     "Squash___Powdery_mildew": "Apply fungicides, space plants for air circulation, and
avoid overhead watering.",
069     "Strawberry___healthy": "No disease detected. Maintain regular care.",
070     "Strawberry___Leaf_scorch": "Remove infected leaves, improve air flow, and apply
fungicides if necessary.",
071     "Tomato___Bacterial_spot": "Apply copper sprays, ensure good air circulation, and
avoid overhead watering.",
072     "Tomato___Early_blight": "Prune lower leaves, apply fungicides, and practice crop
rotation.",
073     "Tomato___healthy": "No disease detected. Continue standard care.",
074     "Tomato___Late_blight": "Apply fungicides immediately and remove infected plant
parts.",
075     "Tomato___Leaf_Mold": "Improve ventilation, avoid overhead watering, and apply
fungicides.",
076     "Tomato___Septoria_leaf_spot": "Remove diseased leaves, mulch around base, and apply
fungicides.",
077     "Tomato___Spider_mites Two-spotted_spider_mite": "Use insecticidal soap, neem oil,
or introduce predatory mites.",
078     "Tomato___Target_Spot": "Ensure good air circulation, avoid overhead irrigation, and
use fungicides.",
079     "Tomato___Tomato_mosaic_virus": "Remove infected plants, sanitize tools, and control
weed hosts.",
080     "Tomato___Tomato_Yellow_Leaf_Curl_Virus": "Control whitefly vectors using
insecticides or reflective mulches.",
081 }
082
083 SYMPTOM_MAP = {
084     "Apple___Apple_scab": [
085         "Olive-green to brown leaf spots",
086         "Velvety lesions on leaf surface",
087         "Premature leaf drop",
088     ],
089     "Apple___Black_rot": [
090         "Dark circular lesions on leaves",
091         "Leaf edge browning",
092         "Black rot spots on fruit in advanced stages",
093     ],
094     "Apple___Cedar_apple_rust": [
095         "Bright orange-yellow spots",
096         "Spore-like structures under leaves",
```

A.6 Configuration - src/config.py (continued)

```
097         "Leaf distortion in severe infection",
098     ],
099     "Apple__healthy": [
100         "Uniform green color",
101         "No visible lesions",
102         "Normal leaf texture",
103     ],
104     "Blueberry__healthy": [
105         "Healthy green leaves",
106         "No spots or discoloration",
107         "Normal fruit development",
108     ],
109     "Cherry_(including_sour)__healthy": [
110         "Healthy green leaves",
111         "No signs of mildew",
112         "Normal leaf texture",
113     ],
114     "Cherry_(including_sour)__Powdery_mildew": [
115         "White, powdery fungal patches on leaves",
116         "Curled or distorted leaves",
117         "Premature leaf drop",
118     ],
119     "Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot": [
120         "Rectangular, gray-to-tan lesions",
121         "Lesions restricted by leaf veins",
122         "Blighting of entire leaves",
123     ],
124     "Corn_(maize)__Common_rust": [
125         "Rust-colored pustules on both leaf surfaces",
126         "Yellowing of surrounding tissue",
127         "Premature leaf death in severe cases",
128     ],
129     "Corn_(maize)__healthy": [
130         "Healthy green leaves",
131         "No visible lesions",
132         "Normal plant growth",
133     ],
134     "Corn_(maize)__Northern_Leaf_Blight": [
135         "Large, cigar-shaped, grayish-green lesions",
136         "Lesions turning tan with age",
137         "Blighted leaves starting from the bottom",
138     ],
139     "Grape__Black_rot": [
140         "Small, light brown spots with dark borders",
141         "Black, shriveled fruit mummies",
142         "Black lesions on shoots",
143     ],
144     "Grape__Esca_(Black_Measles)": [
```

A.6 Configuration - src/config.py (continued)

```
145         "Tiger-stripe pattern on leaves",
146         "Small, dark spots on berries",
147         "Sudden vine wilt",
148     ],
149     "Grape___healthy": [
150         "Healthy green leaves",
151         "No visible lesions",
152         "Normal vine growth",
153     ],
154     "Grape___Leaf_blight_(Isariopsis_Leaf_Spot)": [
155         "Dark red to brown spots on leaves",
156         "Irregularly shaped lesions",
157         "Premature leaf fall",
158     ],
159     "Orange___Haunglongbing_(Citrus_greening)": [
160         "Asymmetrical yellowing of leaves",
161         "Misshapen, bitter fruit",
162         "Twig dieback",
163     ],
164     "Peach___Bacterial_spot": [
165         "Small, angular, water-soaked spots",
166         "Shot-hole appearance as tissue falls out",
167         "Dark lesions on fruit",
168     ],
169     "Peach___healthy": [
170         "Healthy green leaves",
171         "No visible lesions",
172         "Normal plant growth",
173     ],
174     "Pepper,_bell___Bacterial_spot": [
175         "Small, brown, water-soaked spots",
176         "Yellowing and dropping of leaves",
177         "Scab-like spots on fruit",
178     ],
179     "Pepper,_bell___healthy": [
180         "Healthy green leaves",
181         "No visible lesions",
182         "Normal plant growth",
183     ],
184     "Potato___Early_blight": [
185         "Dark spots with concentric rings (target board pattern)",
186         "Yellowing around spots",
187         "Lower leaves affected first",
188     ],
189     "Potato___healthy": [
190         "Healthy green leaves",
191         "No visible lesions",
192         "Normal plant growth",
```

A.6 Configuration - src/config.py (continued)

```
193 ],
194 "Potato___Late_blight": [
195     "Large, dark, water-soaked lesions",
196     "White fungal growth on leaf undersides",
197     "Rapid browning and wilting",
198 ],
199 "Raspberry___healthy": [
200     "Healthy green leaves",
201     "No visible lesions",
202     "Normal plant growth",
203 ],
204 "Soybean___healthy": [
205     "Healthy green leaves",
206     "No visible lesions",
207     "Normal plant growth",
208 ],
209 "Squash___Powdery_mildew": [
210     "White powdery spots on leaves and stems",
211     "Leaves turn yellow and die",
212     "Stunted plant growth",
213 ],
214 "Strawberry___healthy": [
215     "Healthy green leaves",
216     "No visible lesions",
217     "Normal plant growth",
218 ],
219 "Strawberry___Leaf_scorch": [
220     "Irregular purple to brown spots",
221     "Spots lack light centers",
222     "Leaves dry up and look scorched",
223 ],
224 "Tomato___Bacterial_spot": [
225     "Small, dark, water-soaked spots on leaves",
226     "Greasy-looking spots on fruit",
227     "Leaf yellowing and drop",
228 ],
229 "Tomato___Early_blight": [
230     "Dark brown spots with concentric rings",
231     "Lower leaves yellow and drop",
232     "Stem lesions",
233 ],
234 "Tomato___healthy": [
235     "Healthy green leaves",
236     "No visible lesions",
237     "Normal plant growth",
238 ],
239 "Tomato___Late_blight": [
240     "Large, irregularly shaped brown patches",
```

A.6 Configuration - src/config.py (continued)

```
241         "White mold on leaf undersides",
242         "Rapid plant decline",
243     ],
244     "Tomato___Leaf_Mold": [
245         "Pale green or yellow spots on upper leaf",
246         "Olive-green mold on undersides",
247         "Leaves curl and wither",
248     ],
249     "Tomato___Septoria_leaf_spot": [
250         "Small, circular spots with gray centers",
251         "Dark margins around spots",
252         "Lower leaves turn yellow and drop",
253     ],
254     "Tomato___Spider_mites Two-spotted_spider_mite": [
255         "Stippling or tiny yellow spots on leaves",
256         "Fine webbing on leaves and stems",
257         "Leaves turn bronze or yellow",
258     ],
259     "Tomato___Target_Spot": [
260         "Small brown spots with yellow halos",
261         "Concentric rings in older lesions",
262         "Premature defoliation",
263     ],
264     "Tomato___Tomato_mosaic_virus": [
265         "Mottled light and dark green leaves",
266         "Stunted plant growth",
267         "Fern-like leaf appearance",
268     ],
269     "Tomato___Tomato_Yellow_Leaf_Curl_Virus": [
270         "Upward curling of leaf margins",
271         "Yellowing of leaf edges",
272         "Severe stunting and reduced fruit",
273     ],
274 }
```

A.7 Upload Page Template - templates/index.html

```
001 <!DOCTYPE html>
002 <html lang="en" data-theme="dark">
003 <head>
004 <meta charset="UTF-8">
005 <meta name="viewport" content="width=device-width, initial-scale=1.0">
006 <title>LeafGuard AI</title>
007 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;
800&display=swap" rel="stylesheet">
008 <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
009 </head>
010 <body>
011 <div class="bg-scene">
012 <div class="orb orb-1"></div>
013 <div class="orb orb-2"></div>
014 <div class="orb orb-3"></div>
015 </div>
016
017 <div class="app">
018 <nav class="navbar">
019 <div class="logo">
020 <div class="logo-icon">
021 <svg viewBox="0 0 24 24" fill="none">
022 <path d="M12 2C6 2 3 8 3 12c0 4 2 8 5 9.5C9 20 12 17 12 17s3 3 4 4.5C19 20 21
16 21 12c0-4-3-10-9-10z" fill="white" opacity="0.9"/>
023 <path d="M12 2v15" stroke="rgba(255,255,255,0.5)" stroke-width="1" stroke-
linecap="round"/>
024 </svg>
025 </div>
026 LeafGuard<span style="color:var(--accent-lime)">AI</span>
027 </div>
028 <div class="nav-actions">
029 <span class="nav-tag">CNN · EfficientNetB0</span>
030 <a href="/dashboard" class="nav-btn" style="text-decoration: none; font-size: 13px;
font-weight: 600; padding: 6px 14px; border-radius: 20px; background: rgba(45,122,58,0.1);
color: var(--accent-green); border: 1px solid var(--accent-green); transition: all 0.3s;">■
Dashboard</a>
031 <a href="/logout" class="nav-btn" style="text-decoration: none; font-size: 13px;
font-weight: 600; padding: 6px 14px; border-radius: 20px; background: rgba(201,64,64,0.1);
color: var(--accent-red); border: 1px solid var(--accent-red); transition: all 0.3s;">Logout</
a>
032 <button class="theme-btn" id="themeBtn" title="Toggle theme" aria-label="Toggle
theme"></button>
033 </div>
034 </nav>
035
036 <!-- Glowing particles background effect -->
037 <div class="particles">
038 <div class="particle" style="top: 20%; left: 10%; animation-delay: 0s;"></div>
039 <div class="particle" style="top: 60%; left: 80%; animation-delay: 2s;"></div>
040 <div class="particle" style="top: 80%; left: 30%; animation-delay: 4s;"></div>
041 <div class="particle" style="top: 30%; left: 70%; animation-delay: 6s;"></div>
042 </div>
043 <style>
044 .particles { position: absolute; top: 0; left: 0; width: 100%; height: 100%; pointer-
events: none; z-index: -1; overflow: hidden; }
045 .particle { position: absolute; width: 6px; height: 6px; background: var(--accent-
lime); border-radius: 50%; box-shadow: 0 0 15px 4px var(--accent-lime); opacity: 0; animation:
floatParticle 8s ease-in-out infinite; }
046 @keyframes floatParticle { 0% { transform: translateY(0) scale(0.5); opacity: 0; }
50% { transform: translateY(-100px) scale(1.5); opacity: 0.8; } 100% { transform: translateY(-
200px) scale(0.5); opacity: 0; } }
047 .nav-btn:hover { background: var(--accent-green) !important; color: white !important;
box-shadow: 0 4px 15px rgba(45,122,58,0.4); transform: translateY(-2px); }
048 </style>
```

A.7 Upload Page Template - templates/index.html (continued)

```
049
050 <section class="hero">
051   <div class="hero-badge"><div class="badge-dot"></div>Powered by Deep Learning</div>
052   <h1>Detect Plant<br><span class="accent">Disease Instantly</span></h1>
053   <p>Upload a leaf photo and get AI-powered disease detection with treatment
recommendations in seconds.</p>
054 </section>
055
056 <div class="main-grid">
057   <div class="card" style="animation-delay:0.1s">
058     <div class="card-title">
059       <div class="card-title-icon">
060         <svg viewBox="0 0 24 24" fill="none" stroke="var(--accent-green)" stroke-
width="2">
061           <path d="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline points="17 8 12
3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/>
062         </svg>
063       </div>
064       Upload Leaf Image
065     </div>
066
067     <div class="upload-zone" id="uploadZone">
068       <input type="file" id="file-input" accept="image/*">
069       <div id="upload-placeholder">
070         <div class="upload-icon">
071           <svg viewBox="0 0 24 24" fill="none" stroke="white" stroke-width="2">
072             <path d="M12 2C6 2 3 8 3 12c0 4 2 8 5 9.5C9 20 12 17 12 17s3 3 4 4.5C19 20
21 16 21 12c0-4-3-10-9-10z"/>
073           </svg>
074         </div>
075         <div class="upload-text">Drop your leaf photo here</div>
076         <div class="upload-sub">or click to browse files</div>
077       </div>
078       <div id="preview-container" style="display:none">
079         <div class="preview-wrap">
080           <img id="preview-img" src="" alt="Preview">
081           <div class="preview-overlay" onclick="clearImage()"><span>Change Image</
span></div>
082         </div>
083       </div>
084     </div>
085
086
087
088     <div class="processing" id="processing">
089       <div class="scan-ring">
090         <div class="scan-inner">
091           <svg viewBox="0 0 24 24" fill="none" stroke="var(--accent-green)" stroke-
width="2">
092             <circle cx="11" cy="11" r="8"/><path d="m21 21-4.35-4.35"/>
093           </svg>
094         </div>
095       </div>
096       <div class="process-steps"><div class="step-dot"></div><div class="step-dot"></
div><div class="step-dot"></div></div>
```

A.7 Upload Page Template - templates/index.html (continued)

```
097     <div class="process-label" id="process-label">Preprocessing with OpenCV...</div>
098 </div>
099
100 <button class="analyze-btn" id="analyzeBtn" disabled>
101     <div class="btn-shine"></div>
102     Analyze Leaf Disease
103 </button>
104 </div>
105
106 <div class="card" style="animation-delay:0.2s">
107     <div id="diseases-panel">
108         <div class="card-title">Ready for Analysis</div>
109         <div class="diseases-list">
110             <div class="disease-item"><span class="disease-item-name">Upload an image and
Click Analyze for real model prediction.</span></div>
111         </div>
112     </div>
113
114     <div class="result-section" id="result-section">
115         <div class="card-title">Analysis Result</div>
116         <div class="disease-header">
117             <div class="disease-sev" id="res-sev">■</div>
118             <div>
119                 <div class="disease-name" id="res-name">Disease Name</div>
120                 <div class="disease-plant" id="res-plant">Detected on: Plant</div>
121             </div>
122         </div>
123
124         <div class="conf-wrap">
125             <div class="conf-label">
126                 <span>Confidence Score</span>
127                 <span class="conf-pct" id="res-conf">0%</span>
128             </div>
129             <div class="conf-bar-bg"><div class="conf-bar-fill" id="conf-fill"></div></div>
130         </div>
131
132         <div class="info-grid">
133             <div class="info-chip"><div class="info-chip-label">Severity</div><div
class="info-chip-val" id="res-severity"></div></div>
134             <div class="info-chip"><div class="info-chip-label">Model</div><div
class="info-chip-val" id="res-model">EfficientNetB0</div></div>
135             <div class="info-chip"><div class="info-chip-label">Processing</div><div
class="info-chip-val" id="res-processing">OpenCV + CNN</div></div>
136             <div class="info-chip"><div class="info-chip-label">Dataset</div><div
class="info-chip-val" id="res-dataset">PlantVillage</div></div>
137         </div>
138
139         <div class="section-head">Symptoms Detected</div>
140         <div class="symptom-list" id="symptoms-list"></div>
141
142         <div class="section-head">Treatment Protocol</div>
143         <div class="treatment-steps" id="treatment-list"></div>
144     </div>
```

A.7 Upload Page Template - templates/index.html (continued)

```
145     <hr class="divider">
146     <button class="analyze-btn" onclick="resetResult()" style="margin-top:0">
147       <div class="btn-shine"></div>
148       Analyze Another Leaf
149     </button>
150   </div>
151 </div>
152 </div>
153 </div>
154
155 <script>
156 const theme = document.documentElement;
157 let selectedFile = null;
158 let savedTheme = localStorage.getItem('theme');
159 let isDark = savedTheme !== 'light';
160
161 if (!isDark) {
162   theme.setAttribute('data-theme', '');
163 }
164
165 document.getElementById('themeBtn').addEventListener('click', () => {
166   isDark = !isDark;
167   theme.setAttribute('data-theme', isDark ? 'dark' : '');
168   localStorage.setItem('theme', isDark ? 'dark' : 'light');
169 });
170
171 const uploadZone = document.getElementById('uploadZone');
172 const fileInput = document.getElementById('file-input');
173 const analyzeBtn = document.getElementById('analyzeBtn');
174 const previewContainer = document.getElementById('preview-container');
175 const uploadPlaceholder = document.getElementById('upload-placeholder');
176 const previewImg = document.getElementById('preview-img');
177
178 uploadZone.addEventListener('click', e => {
179   if (!e.target.closest('#preview-container')) fileInput.click();
180 });
181 uploadZone.addEventListener('dragover', e => { e.preventDefault(); uploadZone.classList.add('dragging'); });
182 uploadZone.addEventListener('dragleave', () => uploadZone.classList.remove('dragging'));
183 uploadZone.addEventListener('drop', e => {
184   e.preventDefault();
185   uploadZone.classList.remove('dragging');
186   const f = e.dataTransfer.files[0];
187   if (f && f.type.startsWith('image/')) loadImage(f);
188 });
189 fileInput.addEventListener('change', () => {
190   if (fileInput.files[0]) loadImage(fileInput.files[0]);
191 });
192
```

A.7 Upload Page Template - templates/index.html (continued)

```
193 function loadImage(file) {
194     selectedFile = file;
195     const reader = new FileReader();
196     reader.onload = e => {
197         previewImg.src = e.target.result;
198         uploadPlaceholder.style.display = 'none';
199         previewContainer.style.display = 'block';
200         analyzeBtn.disabled = false;
201     };
202     reader.readAsDataURL(file);
203 }
204
205 function clearImage() {
206     selectedFile = null;
207     fileInput.value = '';
208     previewImg.src = '';
209     previewContainer.style.display = 'none';
210     uploadPlaceholder.style.display = 'block';
211     analyzeBtn.disabled = true;
212 }
213
214 async function loadSample(url, filename) {
215     try {
216         const response = await fetch(url);
217         const blob = await response.blob();
218         const file = new File([blob], filename, { type: blob.type });
219         loadImage(file);
220     } catch (error) {
221         console.error("Failed to load sample image", error);
222     }
223 }
224
225 const processLabels = [
226     'Preprocessing with OpenCV...',
227     'Segmenting leaf from background...',
228     'Running through EfficientNetB0...',
229     'Computing confidence scores...',
230     'Generating treatment report...'
231 ];
232
233 async function runAnalysis() {
234     if (!selectedFile) return;
235     const proc = document.getElementById('processing');
236     const lbl = document.getElementById('process-label');
237     analyzeBtn.style.display = 'none';
238     proc.classList.add('active');
239
240     let i = 0;
```

A.7 Upload Page Template - templates/index.html (continued)

```
241 const iv = setInterval(() => {
242   lbl.textContent = processLabels[i % processLabels.length];
243   i += 1;
244 }, 500);
245
246 try {
247   const formData = new FormData();
248   formData.append('leaf_image', selectedFile);
249   const res = await fetch('/predict', { method: 'POST', body: formData });
250   const data = await res.json();
251   if (!res.ok) {
252     alert(data.error || 'Prediction failed.');
```

```
253   return;
254   }
255   showResult(data);
256 } catch (err) {
257   alert('Server error while predicting.');
```

```
258 } finally {
259   clearInterval(iv);
260   proc.classList.remove('active');
261   analyzeBtn.style.display = 'block';
262 }
263 }
264
265 analyzeBtn.addEventListener('click', runAnalysis);
266
267 function showResult(data) {
268   document.getElementById('diseases-panel').style.display = 'none';
269   const rs = document.getElementById('result-section');
270   rs.classList.add('active');
```

```
271
272   const sevMap = { high: '■', med: '■', low: '■' };
273   const sevEl = document.getElementById('res-sev');
274   sevEl.textContent = sevMap[data.severity_class] || '■';
275   sevEl.className = 'disease-sev sev-' + (data.severity_class || 'med');
```

```
276
277   document.getElementById('res-name').textContent = data.disease_name;
278   document.getElementById('res-plant').textContent = 'Detected on: ' + data.plant_name;
279   document.getElementById('res-conf').textContent = data.confidence + '%';
280   document.getElementById('res-severity').textContent = data.severity;
281   document.getElementById('res-model').textContent = data.model;
282   document.getElementById('res-processing').textContent = data.processing;
283   document.getElementById('res-dataset').textContent = data.dataset;
```

```
284
285   setTimeout(() => {
286     document.getElementById('conf-fill').style.width = data.confidence + '%';
287   }, 80);
288 }
```

A.7 Upload Page Template - templates/index.html (continued)

```
289     const sympEl = document.getElementById('symptoms-list');
290     sympEl.innerHTML = (data.symptoms || []).map(s => `<span class="symptom-tag">${s}</span>`).join('');
291
292     const treatEl = document.getElementById('treatment-list');
293     treatEl.innerHTML = (data.treatment || []).map((t,i) =>
294       `<div class="treatment-step"><div class="step-num">${i+1}</div><div class="step-
295       text">${t}</div></div>`.
296     ).join('');
297   }
298   function resetResult() {
299     document.getElementById('result-section').classList.remove('active');
300     document.getElementById('diseases-panel').style.display = 'block';
301     document.getElementById('conf-fill').style.width = '0';
302     clearImage();
303     analyzeBtn.style.display = 'block';
304     document.getElementById('processing').classList.remove('active');
305   }
306 </script>
307 </body>
308 </html>
```

A.8 Dashboard Template - templates/dashboard.html

```
001 <!DOCTYPE html>
002 <html lang="en" data-theme="dark">
003 <head>
004 <meta charset="UTF-8">
005 <meta name="viewport" content="width=device-width, initial-scale=1.0">
006 <title>Dashboard - LeafGuard AI</title>
007 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;
800&display=swap" rel="stylesheet">
008 <link rel="stylesheet" href="{ { url_for('static', filename='style.css') } }">
009 <style>
010 .dashboard-container { max-width: 1200px; margin: 0 auto; padding: 20px; position:
relative; z-index: 1; }
011 .stats-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(250px,
1fr)); gap: 20px; margin-bottom: 30px; }
012 .stat-card { background: var(--bg-card); border: 1px solid var(--border); border-
radius: 20px; padding: 24px; box-shadow: var(--shadow-card); text-align: center; position:
relative; overflow: hidden; }
013 .stat-card::before { content: ''; position: absolute; top: -50%; left: -50%; width:
200%; height: 200%; background: radial-gradient(circle, rgba(126,185,63,0.1) 0%, transparent
70%); z-index: 0; }
014 .stat-value { font-family: 'Inter', sans-serif; font-size: 48px; font-weight: 800;
color: var(--accent-green); position: relative; z-index: 1; text-shadow: 0 0 20px rgba(45,122,
58,0.3); }
015 .stat-label { font-size: 15px; color: var(--text-secondary); margin-top: 8px; font-
weight: 500; position: relative; z-index: 1; }
016 .table-container { background: var(--bg-card); border: 1px solid var(--border); border-
radius: 20px; padding: 24px; box-shadow: var(--shadow-card); overflow-x: auto; }
017 .styled-table { width: 100%; border-collapse: collapse; text-align: left; }
018 .styled-table th, .styled-table td { padding: 16px; border-bottom: 1px solid var(--
border); }
019 .styled-table th { font-family: 'Inter', sans-serif; font-weight: 700; color: var(--
text-secondary); font-size: 14px; text-transform: uppercase; letter-spacing: 1px; }
020 .styled-table tbody tr:hover { background: var(--tag-bg); transition: background 0.3s
ease; }
021 .styled-table td { color: var(--text-primary); font-size: 15px; }
022 .nav-btn { text-decoration: none; font-size: 14px; font-weight: 600; padding: 8px 16px;
border-radius: 20px; background: linear-gradient(135deg, var(--accent-green), var(--accent-
lime)); color: white; border: none; box-shadow: 0 4px 15px rgba(45,122,58,0.3); transition:
transform 0.2s, box-shadow 0.2s; }
023 .nav-btn:hover { transform: translateY(-2px); box-shadow: 0 6px 20px rgba(45,122,58,0.
4); }
024 .severity-badge { display: inline-block; padding: 4px 10px; border-radius: 12px; font-
size: 12px; font-weight: 600; text-transform: uppercase; }
025 .sev-high { background: rgba(201,64,64,0.15); color: var(--accent-red); }
026 .sev-med { background: rgba(212,160,23,0.15); color: var(--accent-yellow); }
027 .sev-low { background: rgba(45,122,58,0.15); color: var(--accent-green); }
028 </style>
029 </head>
030 <body>
031 <div class="bg-scene">
032 <div class="orb orb-1"></div>
033 <div class="orb orb-2"></div>
034 <div class="orb orb-3"></div>
035 </div>
036
037 <div class="dashboard-container">
038 <nav class="navbar">
039 <div class="logo">
040 <div class="logo-icon">
041 <svg viewBox="0 0 24 24" fill="none">
042 <path d="M12 2C6 2 3 8 3 12c0 4 2 8 5 9.5C9 20 12 17 12 17s3 3 4 4.5C19 20 21
16 21 12c0-4-3-10-9-10z" fill="white" opacity="0.9"/>
043 <path d="M12 2v15" stroke="rgba(255,255,255,0.5)" stroke-width="1" stroke-
linecap="round"/>
044 </svg>
045 </div>
046 LeafGuard<span style="color:var(--accent-lime)">AI</span>
047 </div>
048 <div class="nav-actions">
```

A.8 Dashboard Template - templates/dashboard.html (continued)

```
049     <a href="/" class="nav-btn"><- Back to Scanner</a>
050     <a href="/logout" class="nav-btn" style="background: linear-gradient(135deg, var(--
accent-red), #e53935);">Logout</a>
051     <button class="theme-btn" id="themeBtn" title="Toggle theme" aria-label="Toggle
theme"></button>
052   </div>
053 </nav>
054
055 <section class="hero" style="padding: 20px 0 40px;">
056   <h1>Prediction <span class="accent">Dashboard</span></h1>
057   <p>Monitor and analyze all historical plant disease predictions and trends.</p>
058 </section>
059
060 <div class="stats-grid">
061   <div class="stat-card">
062     <div class="stat-value">{{ stats.total_predictions }}</div>
063     <div class="stat-label">Total Predictions</div>
064   </div>
065   {% if stats.top_diseases %}
066   <div class="stat-card">
067     <div class="stat-value" style="font-size: 28px; line-height: 48px; color: var(--
accent-red);">
068       {{ stats.top_diseases[0][0].split('___')[-1].replace('_', ' ') }}
069     </div>
070     <div class="stat-label">Most Common Issue ({{ stats.top_diseases[0][1] }} cases)</
div>
071   </div>
072   <div class="stat-card">
073     <div class="stat-value" style="font-size: 36px; color: var(--accent-lime);">
074       {% set ns = namespace(healthy_count=0) %}
075       {% for d in stats.top_diseases %}
076         {% if 'healthy' in d[0].lower() %}
077           {% set ns.healthy_count = d[1] %}
078         {% endif %}
079       {% endfor %}
080       {{ (ns.healthy_count / stats.total_predictions * 100)|round(1) if stats.
total_predictions > 0 else 0 }}%
081     </div>
082     <div class="stat-label">Healthy Plant Rate</div>
083   </div>
084   {% else %}
085   <div class="stat-card">
086     <div class="stat-value" style="font-size: 28px; line-height: 48px;">No Data</div>
087     <div class="stat-label">Most Common Issue</div>
088   </div>
089   <div class="stat-card">
090     <div class="stat-value" style="font-size: 36px;">0%</div>
091     <div class="stat-label">Healthy Plant Rate</div>
092   </div>
093   {% endif %}
094 </div>
095
096 <div class="table-container">
```

A.8 Dashboard Template - templates/dashboard.html (continued)

```
097     <div class="card-title" style="margin-bottom: 20px;">
098         <div class="card-title-icon">
099             <svg viewBox="0 0 24 24" fill="none" stroke="var(--accent-green)" stroke-
width="2">
100                 <path d="M14 2H6a2 2 0 0 2 2v16a2 2 0 0 2 2h12a2 2 0 0 2 2V8z"></
path><polyline points="14 2 14 8 20 8"></polyline><line x1="16" y1="13" x2="8" y2="13"></
line><line x1="16" y1="17" x2="8" y2="17"></line><polyline points="10 9 9 9 8 9"></polyline>
101             </svg>
102         </div>
103         Recent Scan History
104     </div>
105     <table class="styled-table">
106         <thead>
107             <tr>
108                 <th>Date & Time</th>
109                 <th>Image</th>
110                 <th>Plant</th>
111                 <th>Disease Detected</th>
112                 <th>Confidence</th>
113                 <th>Severity</th>
114             </tr>
115         </thead>
116         <tbody>
117             {% for log in predictions %}
118                 {% set plant_parts = log.disease_name.split('___') %}
119                 {% set plant = plant_parts[0].replace('_', ' ') %}
120                 {% set disease = plant_parts[1].replace('_', ' ') if plant_parts|length > 1 else
log.disease_name %}
121                 {% set is_healthy = 'healthy' in log.disease_name.lower() %}
122                 {% set sev_class = 'sev-low' if is_healthy else ('sev-high' if log.confidence >=
85 else 'sev-med') %}
123                 {% set sev_label = 'Healthy' if is_healthy else ('High' if log.confidence >= 85
else 'Medium') %}
124                 <tr>
125                     <td style="color: var(--text-muted); font-size: 13px;">{{ log.created_at }}</
td>
126                     <td>
127                         <div style="display: flex; align-items: center; gap: 10px;">
128                             <div style="width: 40px; height: 40px; border-radius: 8px; background: url(
'{{ url_for('uploaded_file', filename=log.image_name) }}') center/cover; border: 1px solid
var(--border);"></div>
129                             <span style="font-size: 12px; color: var(--text-muted);">{{ log.image_name[
:15] }}...</span>
130                         </div>
131                     </td>
132                     <td style="font-weight: 500;">{{ plant }}</td>
133                     <td style="color: {{ 'var(--accent-green)' if is_healthy else 'var(--text-
primary)' }}">{{ disease }}</td>
134                     <td>
135                         <div style="display: flex; align-items: center; gap: 8px;">
136                             <span style="font-weight: 700; color: var(--accent-green);">{{ log.
confidence|round(1) }}%</span>
137                             <div style="width: 50px; height: 4px; background: var(--border); border-
radius: 2px;">
138                                 <div style="width: {{ log.confidence }}%; height: 100%; background: var(-
--accent-green); border-radius: 2px;"></div>
139                             </div>
140                         </td>
141                     <td>
142                         <span class="severity-badge {{ sev_class }}">{{ sev_label }}</span></td>
143                     <td><span class="severity-badge {{ sev_class }}">{{ sev_label }}</span></td>
144                 </tr>
```

A.8 Dashboard Template - templates/dashboard.html (continued)

```
145         {% else %}
146         <tr>
147             <td colspan="6" style="text-align: center; padding: 40px; color: var(--text-
muted);">
148                 <div style="margin-bottom: 10px; opacity: 0.5;">
149                     <svg viewBox="0 0 24 24" width="48" height="48" fill="none"
stroke="currentColor" stroke-width="1">
150                         <path d="M22 12h-4l-3 9L9 3l-3 9H2"></path>
151                     </svg>
152                 </div>
153                 No predictions logged yet. Run a scan to see data here!
154             </td>
155         </tr>
156         {% endfor %}
157     </tbody>
158 </table>
159 </div>
160 </div>
161
162 <script>
163 const theme = document.documentElement;
164 let savedTheme = localStorage.getItem('theme');
165 let isDark = savedTheme !== 'light';
166
167 if (!isDark) {
168     theme.setAttribute('data-theme', '');
169 }
170
171 document.getElementById('themeBtn').addEventListener('click', () => {
172     isDark = !isDark;
173     theme.setAttribute('data-theme', isDark ? 'dark' : '');
174     localStorage.setItem('theme', isDark ? 'dark' : 'light');
175 });
176 </script>
177 </body>
178 </html>
```

A.9 Stylesheet - static/style.css

```
001 :root {
002   --bg-primary: #f0f4f0;
003   --bg-secondary: #ffffff;
004   --bg-card: #ffffff;
005   --text-primary: #111a0f;
006   --text-secondary: #445240;
007   --text-muted: #7a9175;
008   --accent-green: #2d7a3a;
009   --accent-lime: #7eb93f;
010   --accent-emerald: #1a9e5c;
011   --accent-yellow: #d4a017;
012   --accent-red: #c94040;
013   --border: rgba(45,122,58,0.15);
014   --shadow: 0 8px 40px rgba(45,122,58,0.12);
015   --shadow-card: 0 4px 24px rgba(0,0,0,0.08);
016   --orb1: rgba(126,185,63,0.18);
017   --orb2: rgba(45,122,58,0.12);
018   --orb3: rgba(26,158,92,0.1);
019   --upload-border: rgba(45,122,58,0.35);
020   --tag-bg: rgba(45,122,58,0.08);
021   --tag-text: #1a5c24;
022   --pill-bg: rgba(126,185,63,0.12);
023 }
024 [data-theme="dark"] {
025   --bg-primary: #0d1a0e;
026   --bg-secondary: #111f12;
027   --bg-card: #162018;
028   --text-primary: #e8f5e2;
029   --text-secondary: #a8c9a0;
030   --text-muted: #607d5a;
031   --accent-green: #5dbf6e;
032   --accent-lime: #9dd45a;
033   --accent-emerald: #3dcf82;
034   --accent-yellow: #f0c040;
035   --accent-red: #f47070;
036   --border: rgba(93,191,110,0.15);
037   --shadow: 0 8px 40px rgba(0,0,0,0.4);
038   --shadow-card: 0 4px 24px rgba(0,0,0,0.3);
039   --orb1: rgba(93,191,110,0.08);
040   --orb2: rgba(61,207,130,0.06);
041   --orb3: rgba(157,212,90,0.07);
042   --upload-border: rgba(93,191,110,0.3);
043   --tag-bg: rgba(93,191,110,0.1);
044   --tag-text: #9dd45a;
045   --pill-bg: rgba(93,191,110,0.08);
046 }
047 *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
048 body { font-family: 'Inter', sans-serif; background: var(--bg-primary); color: var(--text-primary); min-height: 100vh; overflow-x: hidden; }
```

A.9 Stylesheet - static/style.css (continued)

```
049 .bg-scene { position: fixed; inset: 0; z-index: 0; pointer-events: none; overflow:
hidden; }
050 .orb { position: absolute; border-radius: 50%; animation: floatOrb 18s ease-in-out
infinite; filter: blur(60px); }
051 .orb-1 { width: 500px; height: 500px; top: -120px; left: -80px; background: var(--orb1);
}
052 .orb-2 { width: 400px; height: 400px; top: 40%; right: -100px; background: var(--orb2);
animation-delay: -7s; }
053 .orb-3 { width: 350px; height: 350px; bottom: -80px; left: 30%; background: var(--orb3);
animation-delay: -13s; }
054 @keyframes floatOrb { 0%,100% { transform: translate(0,0) scale(1); } 33% { transform:
translate(30px,-40px) scale(1.05); } 66% { transform: translate(-20px,25px) scale(0.96); } }
055 .app { position: relative; z-index: 1; max-width: 1100px; margin: 0 auto; padding: 0
24px 80px; }
056 .navbar { display: flex; align-items: center; justify-content: space-between; padding:
20px 0; }
057 .logo { display: flex; align-items: center; gap: 10px; font-family: 'Inter', sans-serif;
font-weight: 800; font-size: 22px; }
058 .logo-icon { width: 36px; height: 36px; border-radius: 10px; background: linear-gradient(
135deg, var(--accent-green), var(--accent-lime)); display: flex; align-items: center; justify-
content: center; }
059 .logo-icon svg { width: 20px; height: 20px; }
060 .nav-actions { display: flex; align-items: center; gap: 12px; }
061 .nav-tag { font-size: 12px; font-weight: 500; padding: 5px 12px; border-radius: 20px;
background: var(--tag-bg); color: var(--tag-text); border: 1px solid var(--border); }
062 .theme-btn { width: 42px; height: 24px; border-radius: 12px; border: none; cursor:
pointer; background: var(--border); position: relative; }
063 .theme-btn::after { content: ''; position: absolute; top: 3px; left: 3px; width: 18px;
height: 18px; border-radius: 50%; background: var(--accent-green); transition: transform 0.3s;
}
064 [data-theme="dark"] .theme-btn::after { transform: translateX(18px); }
065 .hero { text-align: center; padding: 40px 0 36px; }
066 .hero-badge { display: inline-flex; align-items: center; gap: 6px; font-size: 12px;
padding: 6px 16px; border-radius: 20px; background: var(--pill-bg); color: var(--accent-
green); border: 1px solid var(--border); margin-bottom: 24px; }
067 .badge-dot { width: 6px; height: 6px; border-radius: 50%; background: var(--accent-
emerald); }
068 .hero h1 { font-family: 'Inter', sans-serif; font-weight: 800; font-size: clamp(36px,
6vw, 64px); line-height: 1.05; letter-spacing: -2px; margin-bottom: 18px; }
069 .hero h1 .accent { color: var(--accent-green); }
070 .hero p { font-size: 17px; color: var(--text-secondary); max-width: 520px; margin: 0
auto 18px; line-height: 1.7; }
071 .main-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 24px; margin-bottom:
24px; }
072 @media (max-width: 720px) { .main-grid { grid-template-columns: 1fr; } }
073 .card { background: var(--bg-card); border: 1px solid var(--border); border-radius: 20px;
padding: 28px; box-shadow: var(--shadow-card); }
074 .card-title { font-family: 'Inter', sans-serif; font-weight: 700; font-size: 15px; color:
var(--text-secondary); margin-bottom: 20px; display: flex; align-items: center; gap: 8px; }
075 .card-title-icon { width: 28px; height: 28px; border-radius: 8px; background: var(--tag-
bg); display: flex; align-items: center; justify-content: center; }
076 .upload-zone { border: 2px dashed var(--upload-border); border-radius: 16px; padding:
40px 20px; text-align: center; cursor: pointer; background: var(--tag-bg); }
077 .upload-zone.dragging { border-color: var(--accent-green); background: var(--pill-bg); }
078 .upload-icon { width: 60px; height: 60px; border-radius: 16px; margin: 0 auto 16px;
background: linear-gradient(135deg, var(--accent-green), var(--accent-lime)); display: flex;
align-items: center; justify-content: center; }
079 .upload-icon svg { width: 28px; height: 28px; }
080 .upload-text { font-family: 'Inter', sans-serif; font-weight: 700; font-size: 16px; margin-
bottom: 6px; }
081 .upload-sub { font-size: 13px; color: var(--text-muted); }
082 #file-input { display: none; }
083 .preview-wrap { position: relative; border-radius: 12px; overflow: hidden; margin-top:
16px; }
084 .preview-wrap img { width: 100%; height: 200px; object-fit: cover; border-radius: 12px;
display: block; }
085 .preview-overlay { position: absolute; inset: 0; background: rgba(0,0,0,0.5); display:
flex; align-items: center; justify-content: center; opacity: 0; transition: opacity 0.2s;
cursor: pointer; border-radius: 12px; }
086 .preview-wrap:hover .preview-overlay { opacity: 1; }
087 .preview-overlay span { color: white; font-size: 13px; font-weight: 500; }
088 .analyze-btn { width: 100%; margin-top: 20px; padding: 15px; border: none; border-radius:
12px; cursor: pointer; font-family: 'Inter', sans-serif; font-weight: 700; font-size: 15px;
background: linear-gradient(135deg, var(--accent-green), var(--accent-lime)); color: white;
position: relative; overflow: hidden; }
089 .analyze-btn.disabled { opacity: 0.5; cursor: not-allowed; }
090 .analyze-btn .btn-shine { position: absolute; top: 0; left: -100%; width: 60%; height:
```

```
100%; background: linear-gradient(90deg, transparent, rgba(255,255,255,0.25), transparent);
animation: shine 2.5s ease infinite; }
091 @keyframes shine { 0%{left:-100%;} 60%,100%{left:150%;} }
092 .processing { display: none; flex-direction: column; align-items: center; gap: 16px;
padding: 30px 0; }
093 .processing.active { display: flex; }
094 .scan-ring { width: 80px; height: 80px; position: relative; border-radius: 50%; }
095 .scan-ring::before { content: ''; position: absolute; inset: 0; border-radius: 50%;
border: 3px solid transparent; border-top-color: var(--accent-green); border-right-color: var(
--accent-lime); animation: spinRing 1s linear infinite; }
096 .scan-ring::after { content: ''; position: absolute; inset: 8px; border-radius: 50%;
border: 2px solid transparent; border-bottom-color: var(--accent-emerald); animation:
spinRing 0.8s linear infinite reverse; }
```

A.9 Stylesheet - static/style.css (continued)

```
097 @keyframes spinRing { to { transform: rotate(360deg); } }
098 .scan-inner { position: absolute; inset: 18px; border-radius: 50%; background: var(--tag-
bg); display: flex; align-items: center; justify-content: center; }
099 .scan-inner svg { width: 22px; height: 22px; }
100 .process-steps { display: flex; gap: 6px; }
101 .step-dot { width: 8px; height: 8px; border-radius: 50%; background: var(--accent-green);
opacity: 0.7; }
102 .process-label { font-size: 13px; color: var(--text-muted); }
103 .result-section { display: none; }
104 .result-section.active { display: block; }
105 .disease-header { display: flex; align-items: flex-start; gap: 14px; margin-bottom: 20px;
}
106 .disease-sev { width: 44px; height: 44px; min-width: 44px; border-radius: 12px; display:
flex; align-items: center; justify-content: center; font-size: 20px; }
107 .sev-high { background: rgba(201,64,64,0.12); }
108 .sev-med { background: rgba(212,160,23,0.12); }
109 .sev-low { background: rgba(45,122,58,0.12); }
110 .disease-name { font-family:'Inter', sans-serif; font-weight:800; font-size:20px; line-
height:1.2; }
111 .disease-plant { font-size:13px; color:var(--text-muted); margin-top:3px; }
112 .conf-wrap { margin-bottom: 20px; }
113 .conf-label { display: flex; justify-content: space-between; font-size: 13px; color: var(
--text-secondary); margin-bottom: 8px; }
114 .conf-pct { font-family:'Inter', sans-serif; font-weight:700; color:var(--accent-green);
}
115 .conf-bar-bg { height: 8px; border-radius: 4px; background: var(--border); overflow:
hidden; }
116 .conf-bar-fill { height: 100%; border-radius: 4px; background: linear-gradient(90deg,
var(--accent-green), var(--accent-lime)); width: 0; transition: width 1.2s; }
117 .info-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-bottom:
20px; }
118 .info-chip { padding: 10px 14px; border-radius: 10px; background: var(--tag-bg); border:
1px solid var(--border); }
119 .info-chip-label { font-size: 10px; text-transform: uppercase; letter-spacing: 0.8px;
color: var(--text-muted); margin-bottom: 4px; }
120 .info-chip-val { font-size: 13px; font-weight: 500; }
121 .section-head { font-family:'Inter', sans-serif; font-weight:700; font-size:13px; color:
var(--text-secondary); margin-bottom:10px; }
122 .symptom-list { display: flex; flex-wrap: wrap; gap: 7px; margin-bottom: 18px; }
123 .symptom-tag { font-size: 12px; padding: 4px 12px; border-radius: 20px; background: rgba(
201,64,64,0.08); color: var(--accent-red); border: 1px solid rgba(201,64,64,0.2); }
124 .treatment-steps { display: flex; flex-direction: column; gap: 8px; }
125 .treatment-step { display: flex; align-items: flex-start; gap: 10px; padding: 10px 14px;
border-radius: 10px; background: var(--bg-secondary); border: 1px solid var(--border); }
126 .step-num { min-width: 22px; height: 22px; border-radius: 6px; background: linear-
gradient(135deg, var(--accent-green), var(--accent-lime)); display: flex; align-items: center;
justify-content: center; font-family:'Inter', sans-serif; font-weight:700; font-size:11px;
color:white; }
127 .step-text { font-size: 13px; color: var(--text-secondary); line-height: 1.5; }
128 .diseases-list { display: flex; flex-direction: column; gap: 8px; }
129 .disease-item { display: flex; align-items: center; gap: 12px; padding: 10px 14px;
border-radius: 10px; background: var(--tag-bg); border: 1px solid var(--border); }
130 .disease-item-name { font-size: 13px; font-weight: 500; }
131 .divider { border: none; border-top: 1px solid var(--border); margin: 28px 0; }
132 @media (max-width: 480px) { .hero h1 { font-size: 32px; letter-spacing:-1px; } .info-
grid { grid-template-columns: 1fr; } }
```

A.10 Training Script - train_efficientnet.py

```
001 from __future__ import annotations
002
003 import json
004 from pathlib import Path
005
006 import numpy as np
007 import tensorflow as tf
008 from sklearn.model_selection import train_test_split
009
010
011 PROJECT_ROOT = Path(__file__).resolve().parent
012 DATA_ROOT = PROJECT_ROOT / "plantvillage dataset"
013 MODELS_DIR = PROJECT_ROOT / "models"
014 MODELS_DIR.mkdir(parents=True, exist_ok=True)
015
016 MODEL_PATH = MODELS_DIR / "plant_disease_efficientnetb0.keras"
017 CLASS_NAMES_PATH = MODELS_DIR / "class_names.json"
018
019 # Always train on the original PlantVillage color images (not segmented/grayscale).
020 DATA_SUBDIR = "color"
021
022 # If True, apply the same OpenCV pipeline as the Flask app on each color image.
023 # It is disabled by default because OpenCV inside tf.data is much slower for training.
024 USE_OPENCV_PREPROCESSING = False
025
026 IMG_SIZE = (224, 224)
027 BATCH_SIZE = 32
028 SEED = 42
029 MAX_IMAGES_PER_CLASS = 200
030 INITIAL_EPOCHS = 3
031 FINE_TUNE_EPOCHS = 2
032
033
034 def main() -> None:
035     train_root = DATA_ROOT / DATA_SUBDIR
036     if not train_root.exists():
037         raise FileNotFoundError(f"Dataset folder not found: {train_root}")
038
039     class_names = sorted([p.name for p in train_root.iterdir() if p.is_dir()])
040     num_classes = len(class_names)
041     CLASS_NAMES_PATH.write_text(json.dumps(class_names, indent=2), encoding="utf-8")
042
043     exts = ("*.jpg", "*.jpeg", "*.png", "*.webp")
044     filepaths: list[str] = []
045     labels: list[int] = []
046     for class_idx, class_name in enumerate(class_names):
047         class_dir = train_root / class_name
048         class_files: list[str] = []
```

A.10 Training Script - train_efficientnet.py (continued)

```
049     for ext in exts:
050         for fp in class_dir.glob(ext):
051             class_files.append(str(fp))
052     rng = np.random.default_rng(SEED + class_idx)
053     class_files = sorted(class_files)
054     if MAX_IMAGES_PER_CLASS and len(class_files) > MAX_IMAGES_PER_CLASS:
055         class_files = sorted(rng.choice(class_files, size=MAX_IMAGES_PER_CLASS,
replace=False).tolist())
056     for fp in class_files:
057         filepaths.append(fp)
058         labels.append(class_idx)
059
060     print(f"Training EfficientNetB0 on {len(filepaths)} images across {num_classes}
classes.")
061     if MAX_IMAGES_PER_CLASS:
062         print(f"Using up to {MAX_IMAGES_PER_CLASS} images per class for this run.")
063
064     filepaths_np = np.array(filepaths)
065     labels_np = np.array(labels)
066
067     train_files, test_files, train_labels, test_labels = train_test_split(
068         filepaths_np, labels_np, test_size=0.15, stratify=labels_np, random_state=SEED
069     )
070     train_files, val_files, train_labels, val_labels = train_test_split(
071         train_files,
072         train_labels,
073         test_size=0.15 / (1.0 - 0.15),
074         stratify=train_labels,
075         random_state=SEED,
076     )
077
078     autotune = tf.data.AUTOTUNE
079
080     if USE_OPENCV_PREPROCESSING:
081         import cv2
082         from src.preprocessing.pipeline import LeafPreprocessor
083
084         preprocessor = LeafPreprocessor()
085
086         def cv2_preprocess_rgb(img_rgb_uint8: np.ndarray) -> np.ndarray:
087             img_bgr = cv2.cvtColor(img_rgb_uint8, cv2.COLOR_RGB2BGR)
088             out_rgb_uint8 = preprocessor.preprocess(img_bgr)
089             return out_rgb_uint8.astype(np.float32) / 255.0
090
091         def load_and_preprocess(path: tf.Tensor, label: tf.Tensor) -> tuple[tf.Tensor,
tf.Tensor]:
092             img_bytes = tf.io.read_file(path)
093             img = tf.image.decode_image(img_bytes, channels=3, expand_animations=False)
094             img = tf.cast(img, tf.uint8)
095             out = tf.numpy_function(cv2_preprocess_rgb, [img], tf.float32)
096             out.set_shape((IMG_SIZE[1], IMG_SIZE[0], 3))
```

A.10 Training Script - train_efficientnet.py (continued)

```
097         return out, tf.cast(label, tf.int32)
098
099     else:
100
101         def load_and_preprocess(path: tf.Tensor, label: tf.Tensor) -> tuple[tf.Tensor,
102 tf.Tensor]:
103             img_bytes = tf.io.read_file(path)
104             img = tf.image.decode_image(img_bytes, channels=3, expand_animations=False)
105             img = tf.image.resize(img, IMG_SIZE)
106             img = tf.image.convert_image_dtype(img, tf.float32)
107             return img, tf.cast(label, tf.int32)
108
109         aug = tf.keras.Sequential(
110             [
111                 tf.keras.layers.RandomFlip("horizontal"),
112                 tf.keras.layers.RandomRotation(0.12),
113                 tf.keras.layers.RandomZoom(0.12),
114             ]
115         )
116
117         def make_ds(files: np.ndarray, lbls: np.ndarray, training: bool) -> tf.data.Dataset:
118             ds = tf.data.Dataset.from_tensor_slices((files, lbls))
119             if training:
120                 ds = ds.shuffle(buffer_size=min(len(files), 2000), seed=SEED,
121 reshuffle_each_iteration=True)
122                 ds = ds.map(load_and_preprocess, num_parallel_calls=autotune)
123                 if training:
124                     ds = ds.map(lambda x, y: (aug(x, training=True), y),
125 num_parallel_calls=autotune)
126                 return ds.batch(BATCH_SIZE).prefetch(autotune)
127
128         ds_train = make_ds(train_files, train_labels, training=True)
129         ds_val = make_ds(val_files, val_labels, training=False)
130         ds_test = make_ds(test_files, test_labels, training=False)
131
132         base_model = tf.keras.applications.EfficientNetB0(
133             input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3),
134             include_top=False,
135             weights="imagenet",
136         )
137         base_model.trainable = False
138
139         inputs = tf.keras.Input(shape=(IMG_SIZE[0], IMG_SIZE[1], 3))
140         x = base_model(inputs, training=False)
141         x = tf.keras.layers.GlobalAveragePooling2D()(x)
142         x = tf.keras.layers.Dropout(0.3)(x)
143         outputs = tf.keras.layers.Dense(num_classes, activation="softmax")(x)
144         model = tf.keras.Model(inputs, outputs)
145
146         model.compile(
147             optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
```

A.10 Training Script - train_efficientnet.py (continued)

```
145         loss=tf.keras.losses.SparseCategoricalCrossentropy(),
146         metrics=["accuracy"],
147     )
148
149     callbacks = [
150         tf.keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=4,
151         restore_best_weights=True),
152         tf.keras.callbacks.ReduceLROnPlateau(monitor="val_accuracy", factor=0.5,
153         patience=2),
154     ]
155
156     model.fit(ds_train, validation_data=ds_val, epochs=INITIAL_EPOCHS,
157     callbacks=callbacks)
158
159     base_model.trainable = True
160     for layer in base_model.layers[:-50]:
161         layer.trainable = False
162
163     model.compile(
164         optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
165         loss=tf.keras.losses.SparseCategoricalCrossentropy(),
166         metrics=["accuracy"],
167     )
168     model.fit(ds_train, validation_data=ds_val, epochs=FINE_TUNE_EPOCHS,
169     callbacks=callbacks)
170
171     test_loss, test_acc = model.evaluate(ds_test)
172     print(f"Test accuracy: {test_acc * 100:.2f}%")
173
174     model.save(MODEL_PATH)
175     print(f"Saved model: {MODEL_PATH}")
176     print(f"Saved classes: {CLASS_NAMES_PATH}")
177
178 if __name__ == "__main__":
179     main()
```